

SOUTHEAST UNIVERSITY

Dhaka, Bangladesh

PROJECT REPORT

Image Filtering, Edge Detection, Morphological Processing and Thresholding

CSE355.5: Image Processing

SUBMITTED BY	
Jihad Mia	2023200000505
Md Fahim Islam	2023200000716
Ismail Hossen	2023200000493
Md.Atikur Rahman	2023200000718
Kazi Azmainur Rahman	2023200000562

SUBMITTED TO
Mahbub Hasan **[MAHB]**
Computer Science & Engineering

Date of Submission: **29th September, 2026**

1. Introduction and Pre-processing

This project applies four topics of CSE355 to a single input image so that each task builds on the previous one: (1) spatial and frequency-domain filtering, (2) edge detection, (3) morphological processing and (4) Otsu thresholding. The code is written in Python (NumPy, Matplotlib, *math*); OpenCV is used only to read and resize the image. The algorithms themselves (grayscale conversion, convolution, DFT/IDFT, PSNR/MSE, Sobel, Laplacian, Canny stages, erosion, dilation, histogram and Otsu) are **implemented from scratch**.

1.1 Methodology

- **Resize:** the input image is resized to 256×256 pixels.
- **Grayscale:** a pixel-by-pixel implementation of the weighted sum below.

$$Gray = 0.299\,R + 0.587\,G + 0.114\,B$$

- **Gaussian noise:** zero-mean noise with σ = 25 is added to the clean grayscale image.
- **Salt-and-pepper noise:** 10% of pixels set to white (salt) and 10% to black (pepper), so about 20% of pixels are corrupted.
- **Reference:** the clean grayscale image is the reference for every PSNR/MSE value and is the input to Task 4.

1.2 Outputs

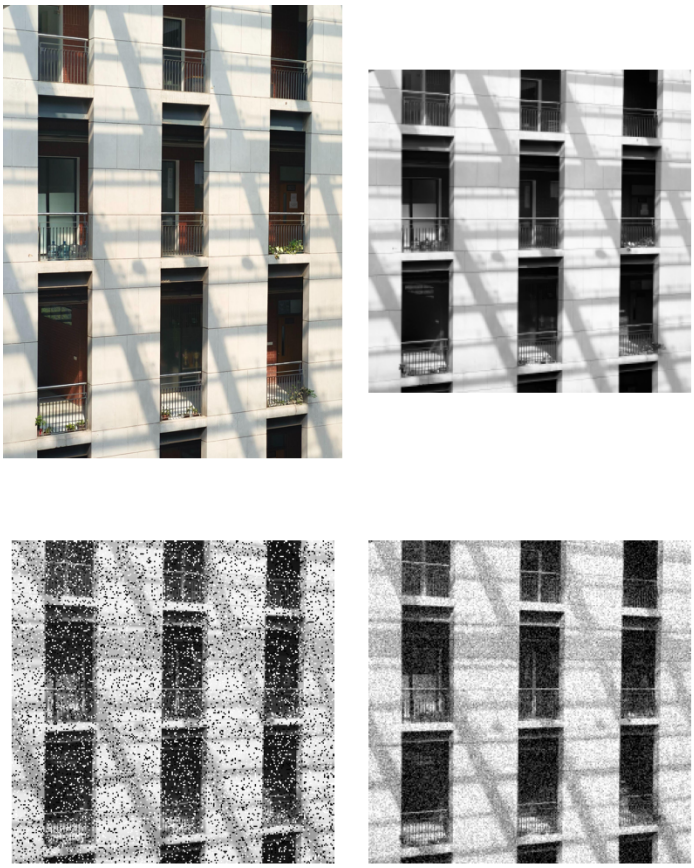


Figure 1: Original (top-left), grayscale (top-right), salt-and-pepper (bottom-left) and Gaussian-noise (bottom-right) images.

2. Task 1 – Image Filtering

2.1 Spatial domain

Methodology. A manual 2-D convolution function (*convolution2d*) zero-pads the image, slides a k×k kernel over every pixel and sums the products of pixel values and kernel weights:

$$g(x, y) = \Sigma\Sigma\, f(x+i, y+j) \cdot h(i, j)$$

Four kernels were applied to both noisy images: **Mean** 3×3 and 5×5 (all weights $1/k^2$) and **Gaussian** 3×3 and 5×5 ($h = \exp(-(x^2+y^2)/2\sigma^2)$, $\sigma = 1$, normalised to sum to 1).

Outputs.

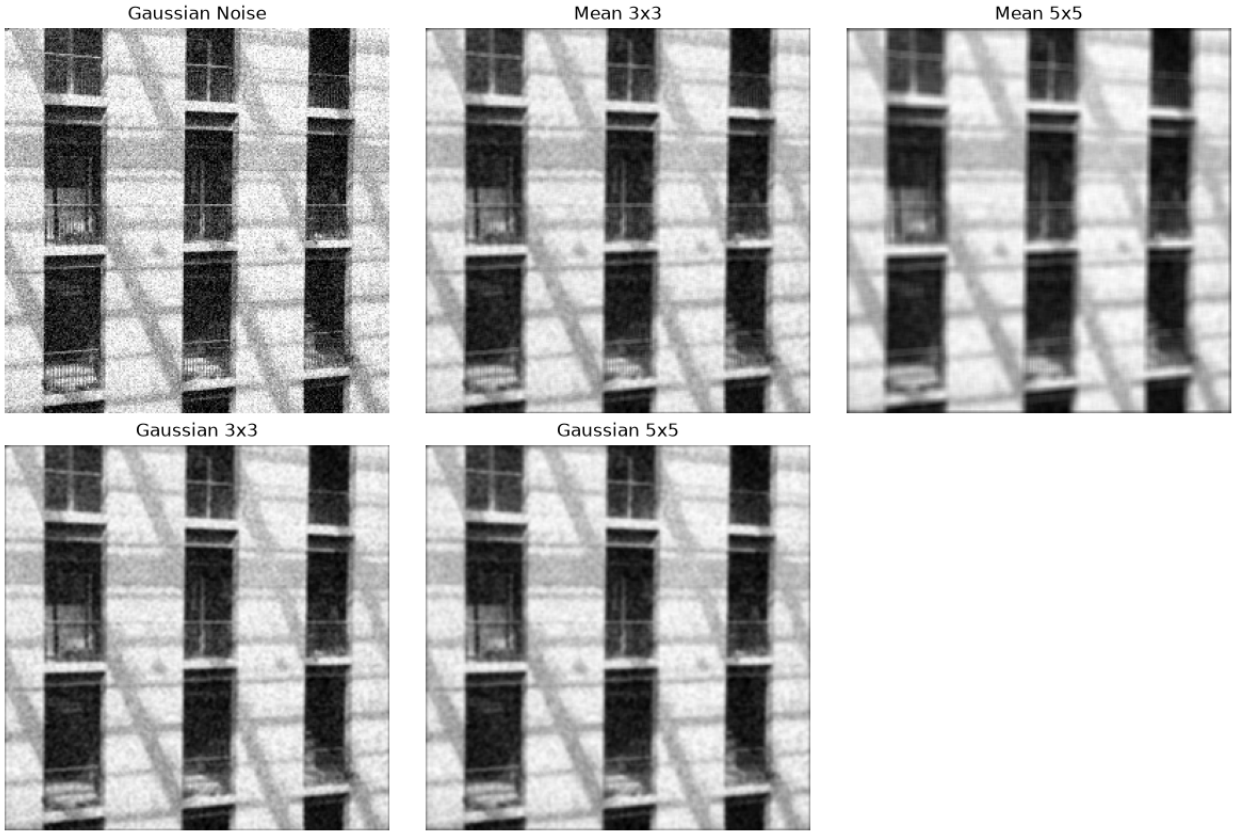


Figure 2: Spatial denoising of the Gaussian-noise image.

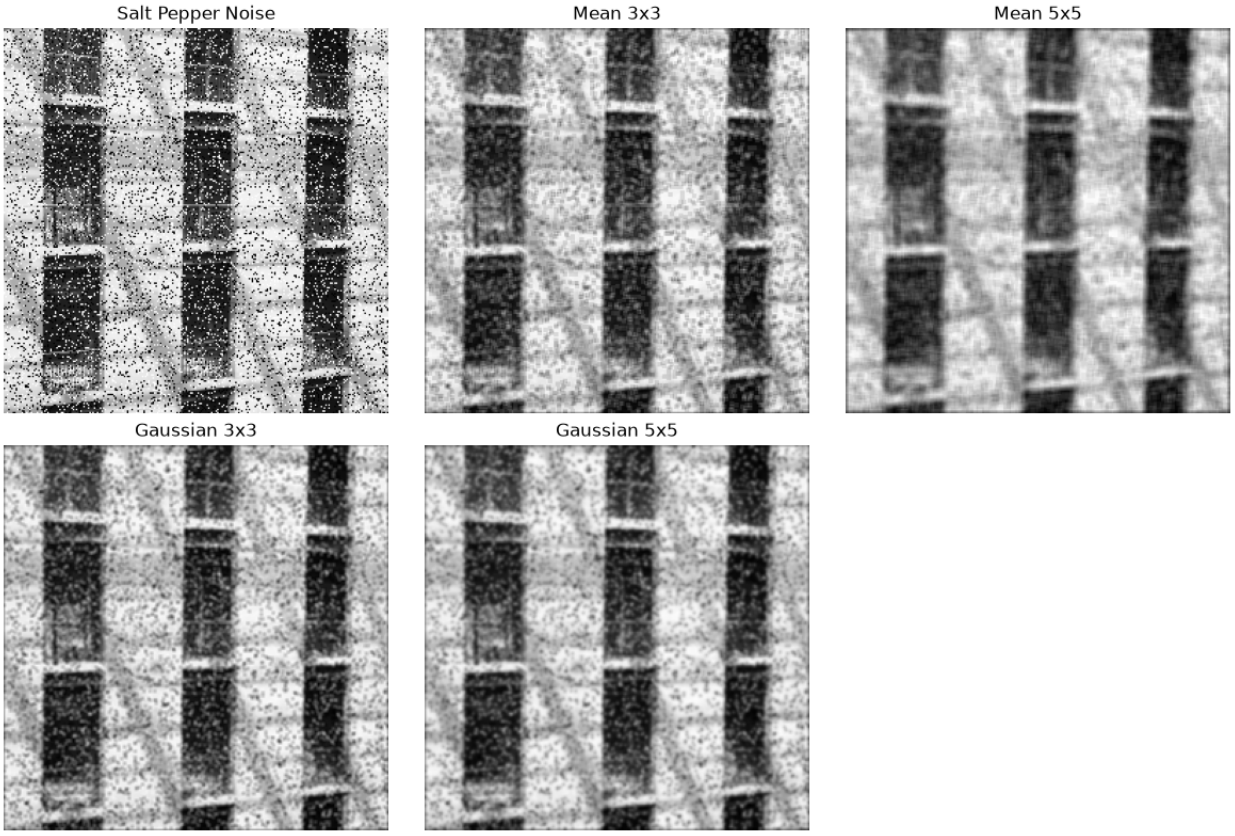


Figure 3: Spatial denoising of the salt-and-pepper image.

Discussion. For Gaussian noise, averaging cancels the zero-mean fluctuations, so both filters reduce the grain, but they also blur edges; the 5×5 mean filter blurs most. For salt-and-pepper noise, the mean and Gaussian filters fail to remove the impulses; they only spread each black or white pixel over its neighbourhood, because a linear filter cannot ignore outliers (a median filter is the usual remedy). Zero padding also darkens the image border slightly.

2.2 Frequency domain

Methodology. The Gaussian-noise image was filtered in five steps:

- A manual separable 2-D DFT (row DFT followed by column DFT, with pre-computed sine/cosine tables): $F(u, v) = \Sigma\Sigma\, f(x, y) \cdot e^{-j2\pi(ux/M + vy/N)}$.
- The spectrum is centred (low frequencies moved to the middle).
- It is multiplied by a **Gaussian low-pass mask** $H = \exp(-D^2/2D_0^2)$ with $\mathbf{D_0 = 30}$.
- The spectrum is shifted back and reconstructed with the manual IDFT: $f = (1/MN)\, \Sigma\Sigma\, G \cdot e^{+j2\pi(\ldots)}$.
- The real part is rounded and clipped to [0, 255]. Spectra are displayed with $\log(1 + |F|)$.

The Gaussian mask was chosen because its smooth cut-off avoids the ringing produced by an ideal filter.

Outputs.

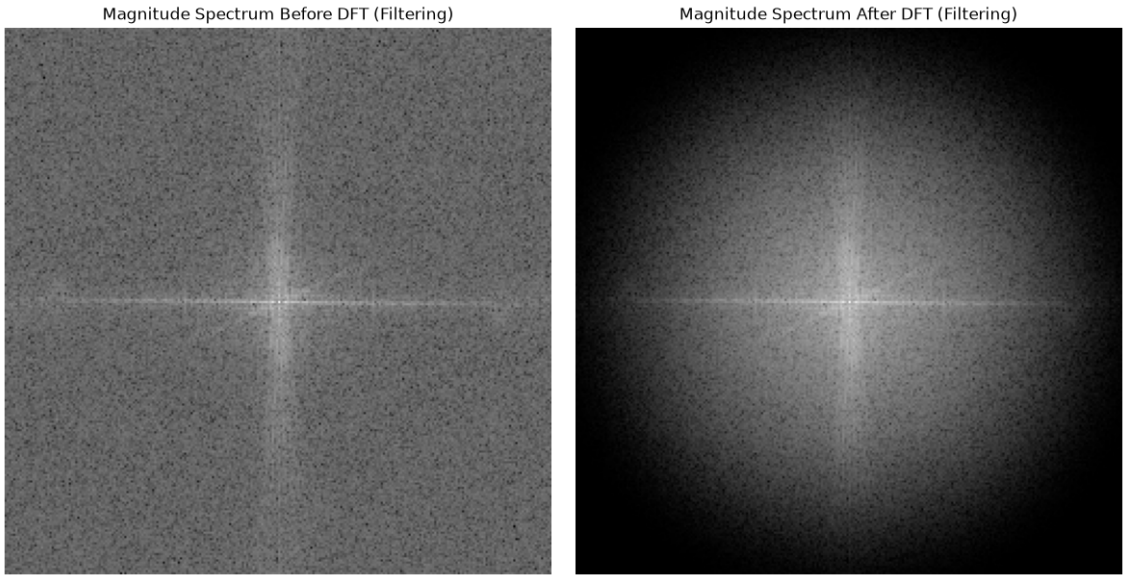


Figure 4a: Magnitude spectrum before and after Gaussian low-pass filtering ($D_0 = 30$).



Figure 4b: IDFT-reconstructed filtered image.

Discussion. Before filtering, the spectrum has a bright centre (image content) and a diffuse halo over all frequencies (the noise, since white noise has similar power at every frequency). After filtering only the central disc survives, so the result is smoother but also softer because real high-frequency detail is removed along with the noise.

2.3 Comparison (PSNR / MSE)

Methodology. Both measures were computed against the clean image for the Gaussian-noise results:

$$MSE = (1/MN)\ \Sigma\Sigma\ [\ f(x, y) - f'(x, y)\ ]^2$$

$$PSNR = 10 \cdot \log_{10}(255^2 / MSE)$$

where f is the clean image and f′ the filtered result.

Method	MSE	PSNR (dB)
Mean 3×3	114.50	27.54
Mean 5×5	143.03	26.58
Gaussian 3×3	108.41	27.78
Gaussian 5×5	**99.74**	**28.14**
Gaussian LPF (frequency, $D_0 = 30$)	115.19	27.52

Table 1: MSE and PSNR of all denoising methods (Gaussian noise).

- **Best:** the spatial Gaussian 5×5 filter (28.14 dB) was selected as the input to Task 2. The frequency-domain filter (27.52 dB) was about 0.6 dB lower.
- **Why they are close:** the Fourier transform of a Gaussian is a Gaussian, so spatial Gaussian smoothing and Gaussian low-pass filtering are essentially the same operation; the gap comes from the choice of D_0, kernel truncation and border handling.
- **Mean vs Gaussian:** the box kernel weights all neighbours equally and has a poor frequency response, so it blurs edges more; the Gaussian weights the centre more. Mean 5×5 was worst because it over-smooths.
- **Trade-off:** in the frequency domain a small D_0 removes more noise but blurs more. For small kernels spatial filtering is cheaper; the manual DFT costs O(MN(M+N)), and an FFT would be used in practice.

3. Task 2 – Edge Detection

Methodology. The input is the Gaussian 5×5 denoised image. The Canny detector was implemented in stages:

- **Gaussian smoothing** (5×5, $\sigma = 1$).
- **Sobel gradients** G_x, G_y, with magnitude $\sqrt{(G_x^2 + G_y^2)}$ and direction $\text{atan2}(G_y, G_x)$.
- **Non-maximum suppression:** a pixel is kept only if it is at least as large as both neighbours along the gradient direction (quantised to 0°, 45°, 90°, 135°), which thins edges to about one pixel.
- **Double thresholding** into strong and weak edges.
- **Hysteresis:** weak pixels 8-connected to a strong edge are promoted, repeated until no change.

A **Laplacian** kernel [0 1 0; 1 −4 1; 0 1 0] was also applied for comparison. Two threshold pairs (low, high) were tested: **(10, 70)** and **(70, 200)**.

Outputs.

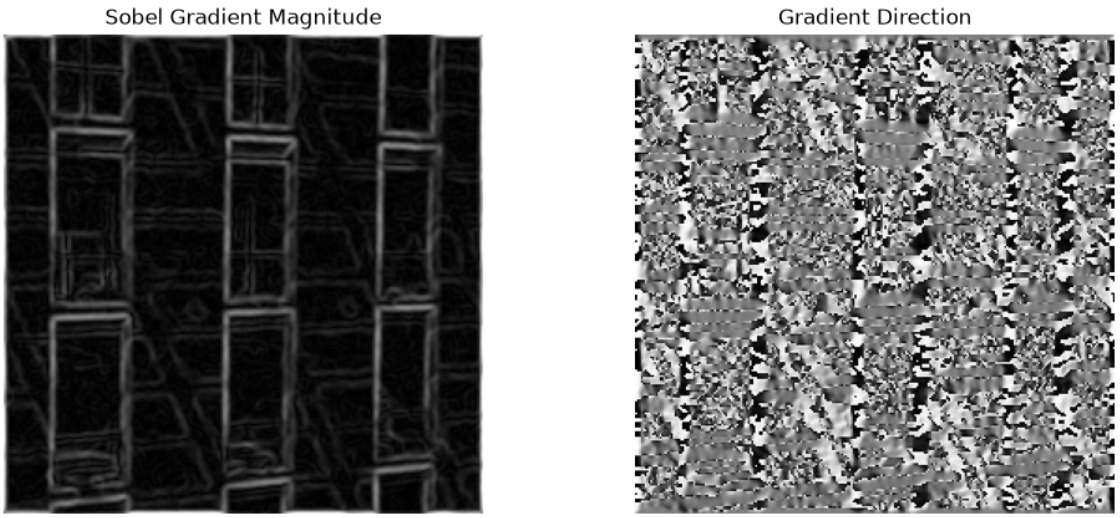


Figure 5a: Sobel gradient magnitude (left) and gradient direction (right).

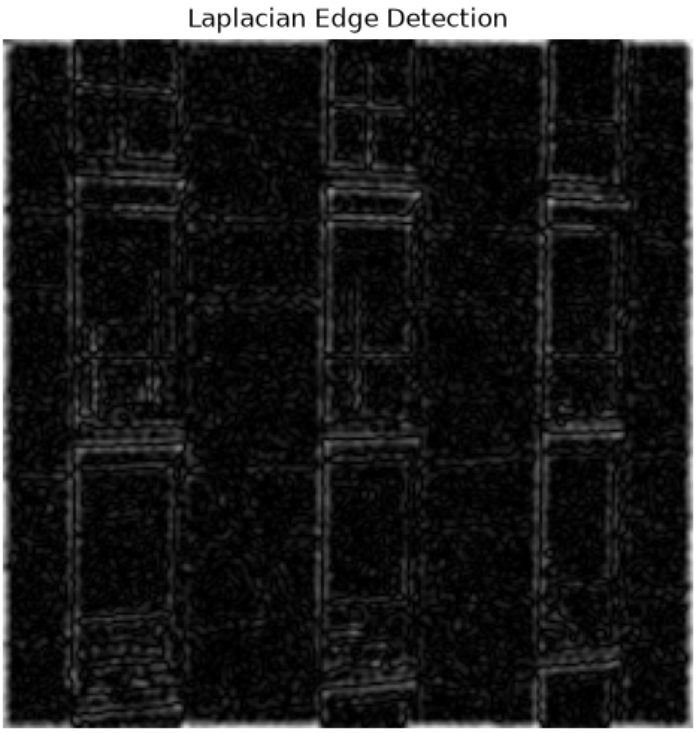


Figure 5b: Laplacian edge response.

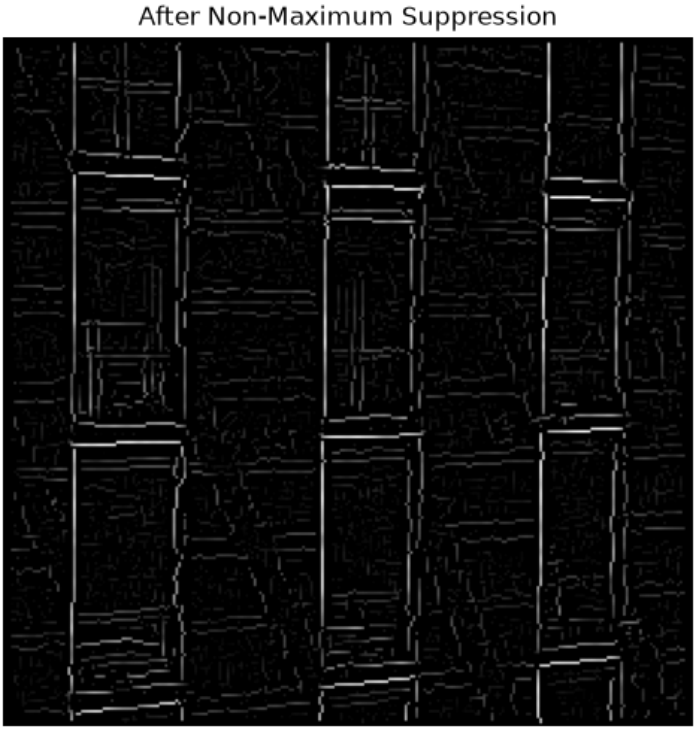


Figure 6a: Result after non-maximum suppression.

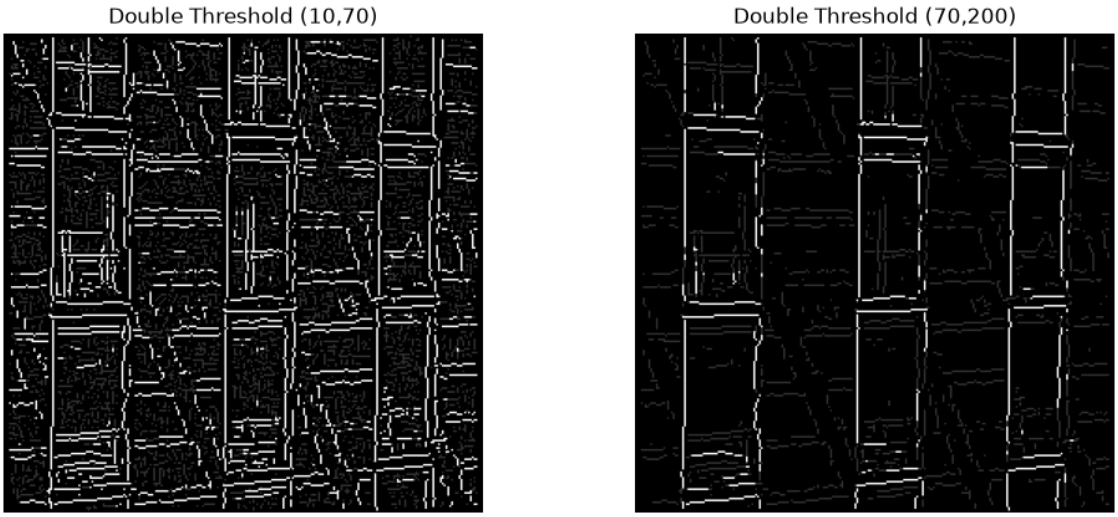


Figure 6b: Double-threshold results for (10, 70) and (70, 200).

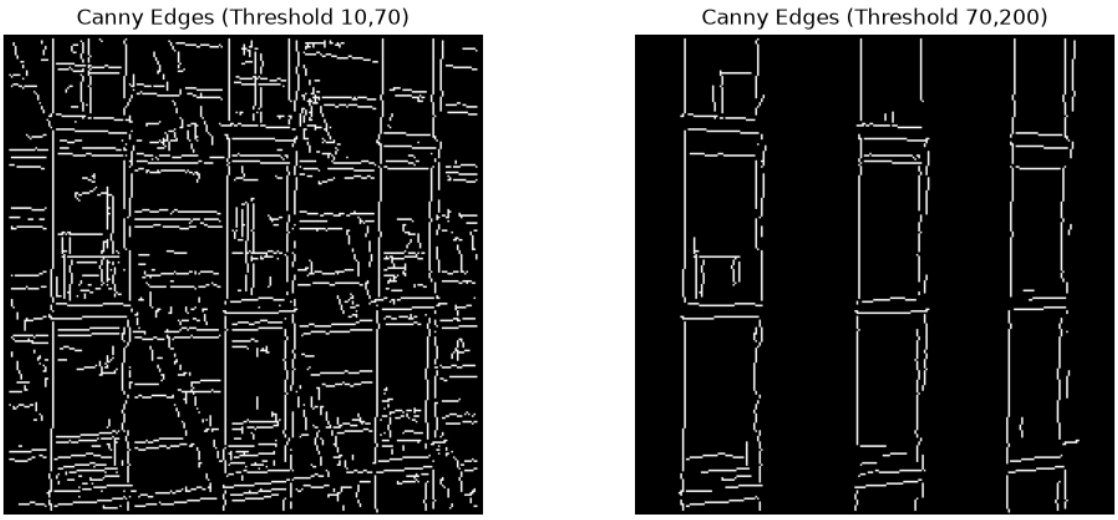


Figure 7: Final Canny edge maps for the two threshold pairs.

Discussion.

- **Threshold choice:** the low pair (10, 70) keeps many weak gradients that touch strong edges, so it captures more fine detail and texture but also more spurious edges from residual noise. The high pair (70, 200) keeps only prominent boundaries, so the map is cleaner and sparser, but faint edges are lost and some contours break.
- **Hysteresis** keeps weak pixels only when connected to strong edges, preserving continuous contours while removing isolated noise.
- **Sobel vs Laplacian:** Sobel (first derivative) gives edge strength and direction, which Canny needs. The Laplacian (second derivative) has no direction, gives double responses at edges and amplifies noise, so smoothing first is essential.

4. Task 3 – Morphological Image Processing

Methodology. The Canny map for (70, 200) was made binary (non-zero → 255, otherwise 0). Erosion and dilation were implemented from scratch with a 3×3 **square** and a 3×3 **cross** structuring element (SE):

- **Erosion** takes the minimum under the SE: a pixel stays white only if the SE fits entirely inside the white region.
- **Dilation** takes the maximum: a pixel turns white if the SE touches any white pixel.

- **Opening** = erosion then dilation; **closing** = dilation then erosion (square SE).

Outputs.

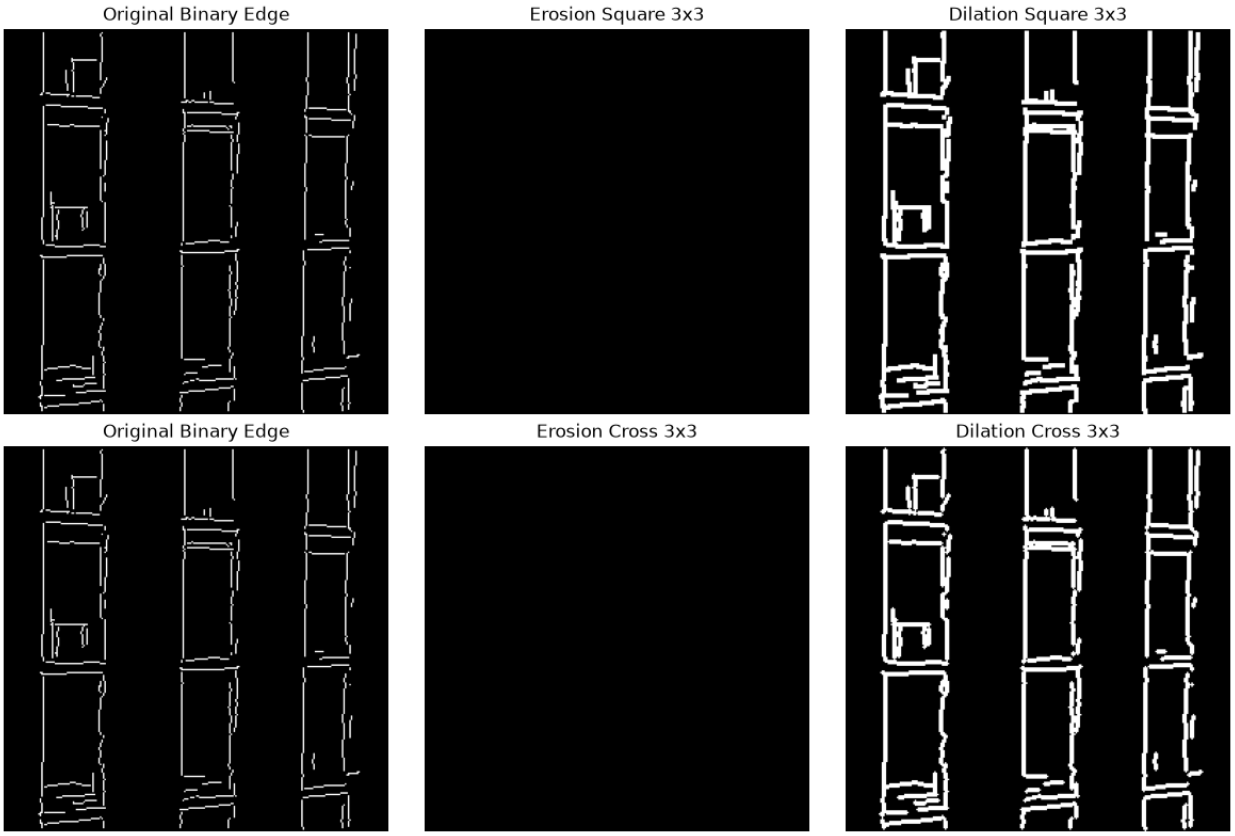


Figure 8: Binary edge map and its erosion/dilation with square and cross SEs.

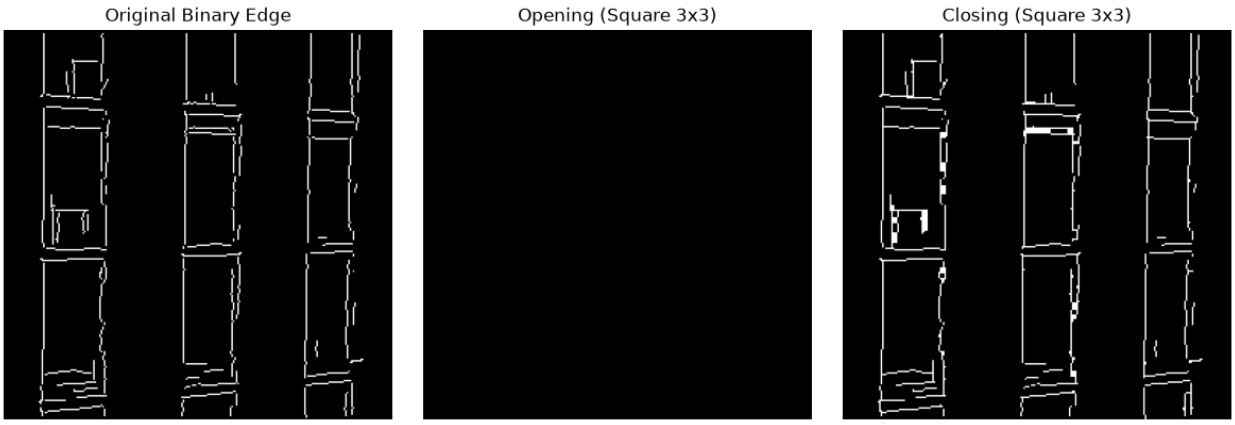


Figure 9: Opening and closing with a 3×3 square SE.

Discussion.

- **Erosion** shrinks the edges and removes isolated pixels. Canny edges are only about one pixel wide, so a 3×3 SE rarely fits inside them and most edge pixels disappear. The square SE erodes more than the cross (8 neighbours required instead of 4).
- **Dilation** thickens edges and bridges gaps of one or two pixels; the square SE thickens more than the cross, but nearby edges may merge.
- **Opening** removes small noise dots but, because it starts with erosion, also breaks thin genuine edges. **Closing** reconnects small gaps and fills tiny holes while keeping thickness close to the original, so it is the better clean-up step for a thin edge map.

5. Task 4 – Otsu's Thresholding

Methodology. Otsu's method was applied to the **original grayscale image**:

- Compute the 256-bin histogram manually.
- For every candidate threshold t = 0…255, split the pixels into background (≤ t) and foreground (> t) with weights ω_0, ω_1 and means μ_0, μ_1 (cumulative sums are updated incrementally).
- Compute the between-class variance for each t:

$$\sigma^2_B(t) = \omega_0(t) \cdot \omega_1(t) \cdot [\, \mu_0(t) - \mu_1(t) \,]^2$$

- Choose T = argmax $\sigma^2_B(t)$.
- Set pixels ≥ T to 255 and the rest to 0.

The code uses pixel counts instead of probabilities, which scales every value by the same constant and does not change the maximiser. **The threshold marked on the histogram (Figure 10) is T = 129**.

Outputs.

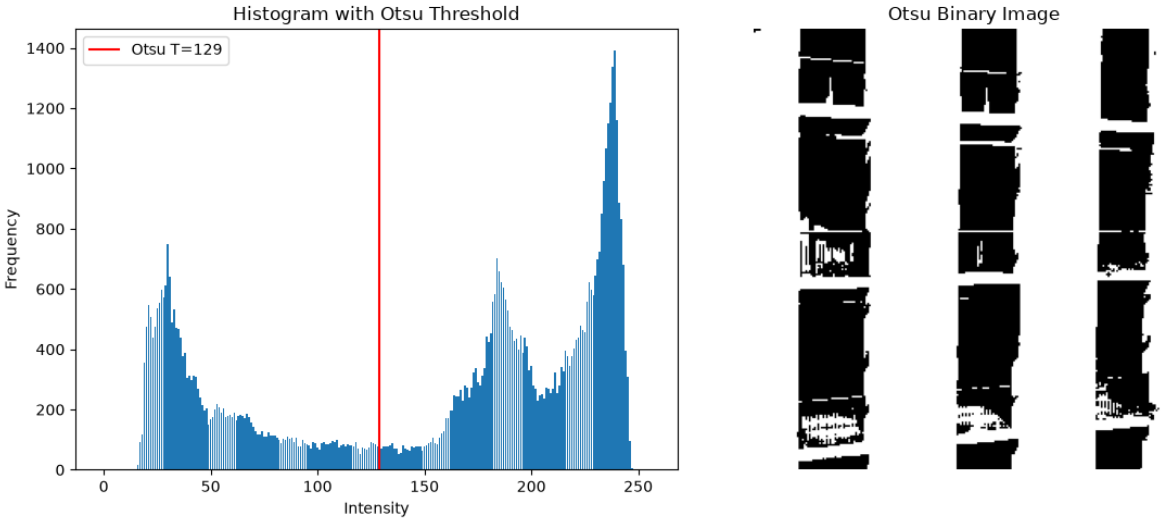


Figure 10: Histogram with the Otsu threshold (T = 129) and the segmented image.

Discussion. Otsu chooses the threshold automatically at the point where the two pixel populations are most separated, and it works best for a roughly bimodal histogram, as here (a dark peak and a bright peak with a shallow valley between them). Its limits are that a single global threshold cannot handle uneven illumination and can be biased when the classes differ greatly in size; adaptive thresholding would suit those cases. The clean image was used because noise reshapes the histogram and can shift the threshold.

6. Conclusion

Spatial Gaussian 5×5 filtering gave the best denoising (28.14 dB), slightly ahead of the frequency-domain Gaussian low-pass filter (27.52 dB); neither linear filter removes salt-and-pepper noise. Canny thresholds control the balance between edge detail and cleanliness; erosion and opening are destructive on thin edge maps while dilation and closing repair gaps; and Otsu's method selected a global threshold of 129 by maximising the between-class variance.

7. Individual Contribution Table

#	Name	Student ID	Contribution
1	Jihad Mia	2023200000505	Lead, Work Distribution and Coding
2	Md Fahim Islam	2023200000716	Coding and Documentation
3	Ismail Hossen	2023200000493	Math and Documentation
4	Md.Atikur Rahman	2023200000718	Math and Documentation
5	Kazi Azmainur Rahman	2023200000562	Test and Validation